

BibliotecaIA

DOCUMENTAÇÃO DO PROJETO DE CONCLUSÃO DE CURSO

LETICIA GUERRA | ITERA360 | 2025

GITHUB: [GITHUB.COM/LETICIA-GUERRA/BIBLIOTECAIA](https://github.com/leticia-guerra/bibliotecaia)

INTRODUÇÃO

1. DESCRIÇÃO GERAL DO PROJETO:

A BibliotecalA é um sistema desenvolvido para ajudar leitores a organizar suas leituras e receber recomendações personalizadas com o uso de Inteligência Artificial. A motivação veio do gosto pessoal pela leitura e da necessidade de ter um lugar organizado para registrar os livros lidos e descobrir novos.

2. OBJETIVOS:

Permitir que o usuário cadastre e organize os livros que leu - Acompanhar estatísticas como total de páginas lidas e avaliação média - Usar Inteligência Artificial para recomendar novas leituras com base no histórico do usuário

3. ESCOPO:

O alcance do projeto:

- Cadastra e organiza livros lidos
- Exibe estatísticas de leitura
- Gera recomendações via IA

Limitações:

- Não tem lista de "quero ler"
- Não salva o histórico de recomendações geradas
- Não exibe capa dos livros
- Não tem autenticação JWT
- Não está publicado em nuvem

VISÃO GERAL DO SISTEMA

1. ARQUITETURA DO SISTEMA:

O projeto foi dividido em camadas para deixar o código mais organizado e facilitar a manutenção. Todas as camadas são importantes e cada uma tem uma responsabilidade específica:

- Api — é a porta de entrada do sistema. Recebe as requisições HTTP e devolve as respostas, mas não faz o trabalho sozinha
 - [BibliotecalA.Api](#)
- Aplicacao — é onde o trabalho acontece de verdade. Contém as regras de negócio, valida os dados e orquestra as operações, como montar o prompt para a IA
 - [BibliotecalA.Aplicacao](#)

- Dominio — contém as entidades puras do sistema como Livro, Usuario e CatalogoLivro
 - [BibliotecalA.Dominio](#)
- Repositorio — é responsável por acessar o banco de dados, usando Entity Framework e Dapper
 - [BibliotecalA.Repositorio](#)
- O frontend foi desenvolvido em React e consome a API REST. O React permite criar componentes reutilizáveis — por exemplo, o CardResumo foi criado uma vez e reutilizado 4 vezes na página Home para exibir diferentes estatísticas de leitura.

2. FUNCIONALIDADES:

Gestão de Usuários

- Cadastro com nome, e-mail, senha e tipo (comum ou administrador)
- Login com validação de credenciais
- Atualização de dados e alteração de senha
- Exclusão lógica

Diário de Leitura

- Cadastro de livros lidos com título, autor, gênero, páginas, data de leitura, avaliação (1-5) e comentário
- Listagem personalizada com busca por título/autor e filtro por gênero
- O usuario tambem consegue editar e excluir livros direto da Home

Dashboard de Estatísticas

- Total de livros lidos, total de páginas, avaliação média e gênero favorito

Recomendações por IA Generativa

- O usuário seleciona um gênero e recebe 3 recomendações personalizadas geradas pelo GPT-4.1
- O histórico de leitura do usuário é usado como contexto para o prompt
- Cada recomendação inclui título, autor, páginas e justificativa personalizada

Painel Administrativo

- Gerenciamento completo do catálogo editorial de livros
- Listagem e gerenciamento de todos os usuários da plataforma

CONFIGURAÇÃO DO AMBIENTE

1. REQUISITOS DE SOFTWARE E HARDWARE:

Itens necessarios:

- *.NET 9*
- *Node.js*

- *SQL Server*
- *Token do GitHub para a IA funcionar*
- *Hardware mínimo*

2. INSTRUÇÕES DE INSTALAÇÃO:

Backend

1. Clonar o repositório:
2. Configurar a string de conexão em `BibliotecaIA.Api/appsettings.json`:
3. Configurar o token do GitHub Models no mesmo arquivo:
4. Executar as migrations para criar o banco de dados:
5. Iniciar a API:

A API estará disponível em `https://localhost:5211` e o Swagger em `/swagger`.

Frontend

1. Acessar a pasta do frontend e instalar dependências:

```
cd bibliotecaia-app
```

```
npm install
```

2. Iniciar a aplicação:

```
npm start
```

A aplicação estará disponível em `http://localhost:3000`.

3. CONFIGURAÇÃO DO AMBIENTE DE DESENVOLVIMENTO:

Para desenvolver o projeto foi utilizado o Visual Studio como editor de código. O banco de dados foi configurado através do repositório. O frontend se conecta ao backend através do endereço `localhost:5211`.

4. DEPENDENCIAS:

Backend:

- Entity Framework Core — para acessar o banco de dados
- Dapper — para executar as Stored Procedures

- Swagger – para documentar a API
- System.Net.Http – para fazer as chamadas HTTP para a API do GitHub Models (IA)

Frontend:

- React Router – para gerenciar as rotas e proteger as páginas
- Bootstrap – para os componentes visuais
- React Icons – para os ícones da interface

DESENVOLVIMENTO

1. ESTRUTURA DO PROJETO:

O projeto está organizado da seguinte forma:

Backend:

- BibliotecalA.Api – camada de apresentação
- BibliotecalA.Aplicacao – camada de aplicação
- BibliotecalA.Dominio – camada de domínio
- BibliotecalA.Repositorio – camada de repositório

Frontend:

- paginas/ – componentes de cada tela
- routes/ – controle de rotas e proteção de páginas
- componentes/ – componentes reutilizáveis como o CardResumo
- utils/ – dados auxiliares como a lista de gêneros

2. DESCRIÇÃO DAS CAMADAS:

Camada de Apresentação (BibliotecalA.Api)

Responsável por receber as requisições do frontend e retornar as respostas. Contém os controllers LivroController, UsuarioController, AlController e CatalogoLivroController . Também configura o acesso do frontend React e o Swagger.

Camada de Aplicação (BibliotecalA.Aplicacao)

Camada de Aplicação (BibliotecalA.Aplicacao) – contém as regras de negócio. É aqui que acontece a validação dos dados, o fluxo das recomendações e a integração com a IA.

Camada de Domínio (BibliotecalA.Dominio)

Contém as entidades puras do sistema sem nenhuma dependência externa. A entidade Livro valida que a avaliação esteja entre 1 e 5. Todas as entidades implementam soft delete através dos métodos

Deletar() e Restaurar() que alteram o campo Ativo.

Camada de Repositório (Biblioteca\A.Repositorio)

Responsável pelo acesso ao banco de dados. Utiliza Entity Framework Core para operações CRUD convencionais e Dapper para executar as Stored Procedures do SQL Server. O LivroSqlRepositorio executa as procedures sp_ListarLivrosPorUsuario, sp_InserirLivroPorUsuario, sp_AtualizarLivroUsuario, sp_ExcluirLivroUsuario e sp_BuscarLivroPorIdEUsuario, além das functions fn_QuantidadeLivrosPorUsuario e fn_TotalPaginasPorUsuario e a view vw_Livros.

API

1. DOCUMENTAÇÃO DA API:

USUARIO

Metodo	Rota	Descrição
POST	UsuarioCriar	Cria um novo usuario
POST	UsuaarioLogin	Autentica e retorna dados do usuário
GET	UsuarioObter	obtem dados de um usuario
GET	UsuarioListar	lista usuarios ativos ou inativos
PUT	UsuarioAtualizar	atualiza nome e email
PUT	UsuarioAlterarSenha	altera a senha
DELETE	UsuarioDeletar	exclusão lodica do usuario
PUT	UsuarioRestaurar	restaura usuario inativo

LIVROS

<i>Metodo</i>	<i>Rotas</i>	<i>Descrição</i>
<i>POST</i>	<i>LivroCriar</i>	<i>cadastra novo livro</i>
<i>GET</i>	<i>LivroObter</i>	<i>obtem um livro por ID</i>
<i>GET</i>	<i>LivroListar</i>	<i>lista todos os livros ativos</i>
<i>GET</i>	<i>ListarPorUsuario</i>	<i>lista livros de um usuario</i>
<i>GET</i>	<i>BuscarPorTitulo</i>	<i>busca livros por titulo</i>
<i>DELETE</i>	<i>LivroDeletar</i>	<i>exclusão logica do livro</i>
<i>PUT</i>	<i>LivroRestaurar</i>	<i>restaura livro excluido</i>

CATALOGOLIVRO

<i>Metodo</i>	<i>Rota</i>	<i>Descrição</i>
<i>POST</i>	<i>CatalogoLivroCriar</i>	<i>Adiciona livro ao catalogo</i>
<i>PUT</i>	<i>CatalogoLivroAtualizar</i>	<i>Atualiza dados do catalogo</i>
<i>GET</i>	<i>CatalogoLivroObter</i>	<i>Obtem livro do catalogo</i>
<i>GET</i>	<i>CatalogoLivroListar</i>	<i>Lista catalogo</i>
<i>GET</i>	<i>CatalogoLivroListarPorGenero</i>	<i>Filtra catalogo por genero</i>
<i>GET</i>	<i>CatalogoLivroBuscarPorTitulo</i>	<i>Busca por titulo</i>

LIVROSQL

GET	LivroSqlListarPorUsuario	Lista os livros da biblioteca de um usuario via SQL
POST	LivroSqlInserirPorUsuario	Insere um livro na biblioteca do usuario via Sql
GET	LivroSqlBuscarPorUsuario	Busca um livro especifico na biblioteca do usuario
PUT	LivroSqlAtualizarPorUsuario	atualiza dados de um livro na biblioteca de um usuario
DELETE	LivroSqlExcluirPorUsuario	Exclusão logica de um livro da biblioteca do usuario via SQL
GET	LivroSqlQuantidadePorUsuario	Quantidade de livros por usuario
GET	LivroSqlTotalPaginasPorUsuario	total de paginas lidas por usuario
GET	LivroSqlListarView	Lista os livros usando view SQL otimizada

AI

POST	AICompletar	Realiza completção de texto via IA
GET	AIRecomendarPorUsuario	Gera recomendações personalizadas de livros via IA

INTERFACE DO USUÁRIO

1. DESCRIÇÃO DAS FUNCIONALIDADES DA INTERFACE:

Login — o usuário faz login ou cria uma nova conta, e depois entra na home. O Adiministrador tambem entra por aqui so que e direcionado para uma outra home.

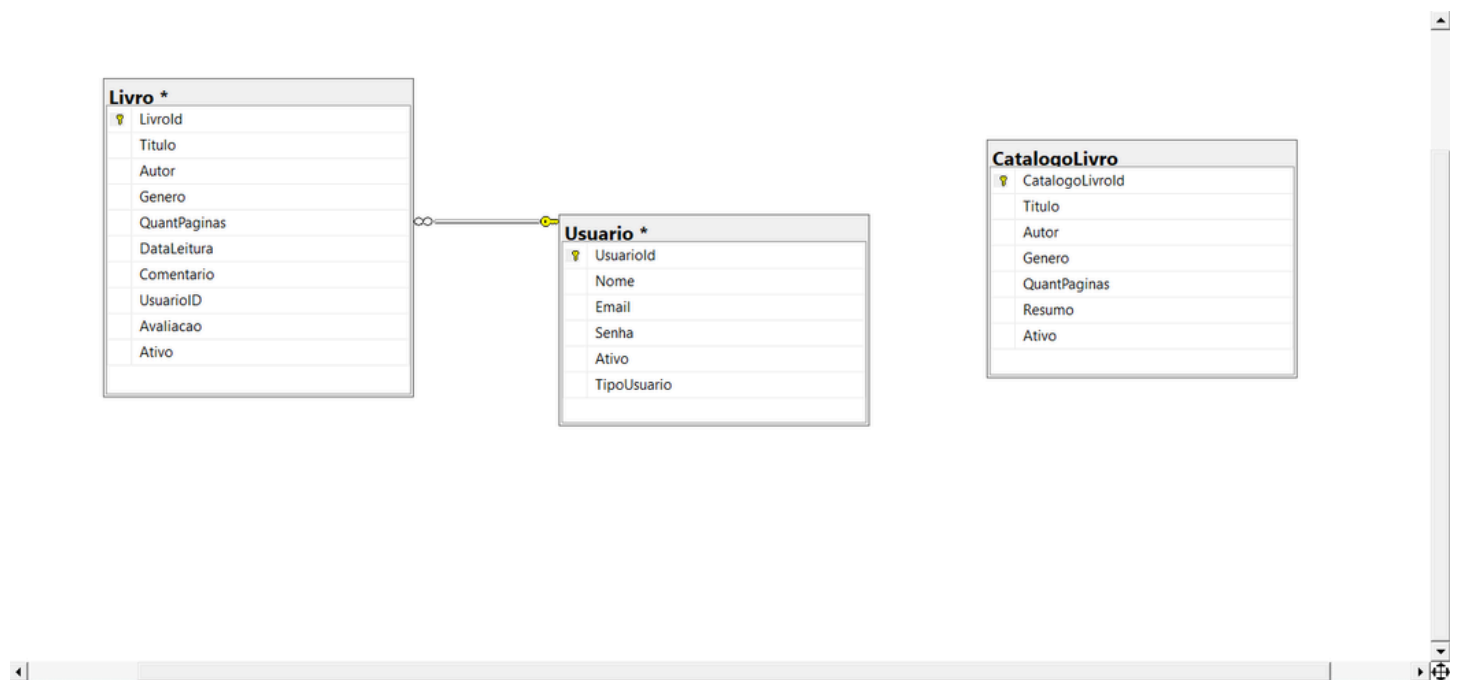
Home — o usuário visualiza os livros lidos onde consegue excluir e editar os livro e tambem fazer a busca por título e o filtro por gênero, quantidade de páginas, gênero favorito e pode buscar recomendações pela IA.

Cadastro de Livro — o usuário cadastra uma nova leitura preenchendo as informações pedidas, como título, autor, quantidade de páginas... Aqui o usuário também consegue entrar quando decide que já leu tal livro que a IA recomendou, e já vem preenchido com título, autor, quantidade de páginas, e só fica sobrando para o usuário cadastrar informações pessoais como data que foi livro, avaliação e algum comentário.

Admin — Nessa página somente administradores conseguem acessar. O administrador escolhe entre gerenciar o catálogo de livros ou os usuários da plataforma.

BANCO DE DADOS

DIAGRAMA DE ENTIDADE-RELACIONAMENTO (ERD):



CONSIDERAÇÕES FINAIS

1. LIÇÕES APRENDIDAS:

A maior dificuldade do projeto foi a integração com a IA Generativa. Foi necessário entender como configurar o token do GitHub, montar o prompt corretamente e fazer a chamada funcionar. O aprendizado veio através de tentativa e erro até conseguir fazer tudo funcionar junto. Outro aprendizado importante foi a arquitetura em camadas, que deixa o código mais organizado e fácil de manter. Cada parte do sistema tem sua responsabilidade, o que facilita encontrar e corrigir problemas.

2. MELHORES PRÁTICAS:

Uma boa prática aprendida foi planejar e estruturar o projeto antes de começar a codar. Definir as funcionalidades, as camadas e o banco de dados com antecedência evita retrabalho durante o desenvolvimento e dor de cabeça.

3. PRÓXIMOS PASSOS:

Para versões futuras do BibliotecalA estão planejadas as seguintes melhorias:

- Autenticação JWT para deixar o login mais seguro
 - Lista de livros que o usuário quer ler futuramente
 - Histórico das recomendações geradas pela IA
 - Exibição da capa dos livros
-

ANEXOS:

Links Úteis:

- Repositório GitHub: <https://github.com/leticia-guerra/biblioteca>
- Swagger: <https://localhost:5211/swagger>

Referências:

- Documentação .NET: <https://docs.microsoft.com/dotnet>
- Documentação React: <https://react.dev>
- Documentação GitHub Models: <https://docs.github.com/en/github-models>
- React Icons: <https://react-icons.github.io>
- W3Schools: <https://w3schools.com>
- Material didático do curso – Itera360

Créditos:

- Projeto desenvolvido por Leticia Guerra como trabalho de conclusão do curso de desenvolvimento full stack .NET + React na Itera360 (2025-2026).

